

ADT Implementations + Performance Shootout

STANDARDIZE YOUR BENCHMARK DRIVER PROGRAM WITH THESE NUMBERS:

```
private static final int WARMUP_OPS = 15_000;  
private static final int MEASURE_OPS = 60_000;
```

Advanced Java: Stacks, Queues, and Priority Queues

Deliverable: Implement multiple ADTs + build a repeatable micro-benchmark suite.
Focus: correctness, Big-O, and real performance (cache locality, shifting, pointer chasing, JIT).

Learning Goals

- Implement ADTs from interfaces: Stack, Queue, PriorityQueue.
- Implement each ADT using different underlying structures (ArrayList vs DLinkedList).
- Implement PriorityQueue 3 ways: sorted ArrayList, sorted DLinkedList, and a binary heap (provided).
- Build a reliable benchmark: warmups, multiple trials, fixed random seed, checksum to prevent dead-code elimination.
- Interpret results: compare observed timings to theoretical complexity.

Starter Code Provided

- Interfaces in `chapter9: Stack, Queue, PriorityQueue`
- Linked list in `my.util: DLinkedList` + `DNode` (includes `removeFirst()`)
- Heap-based PQ: `chapter9.BinaryHeapPriorityQueue` (read-only reference)
- Benchmark skeleton: `benchmark.BenchmarkDriver`

Priority convention: lower integer = higher priority. Example: priority 1 is "more urgent" than priority 5.

What You Must Implement

Part A: Stacks (2 implementations)

- `chapter9.ArrayListStack<T>`
- `chapter9.DLinkedListStack<T>` (use tail as top)

Part B: Queues (2 implementations)

- `chapter9.ArrayListQueue<T>` implemented as a **circular buffer** inside an ArrayList (no $O(n)$ shifting on dequeue)
- `chapter9.DLinkedListQueue<T>` (enqueue at tail, dequeue from head using `removeFirst()`)

Part C: Priority Queues (2 implementations + 1 provided)

- `chapter9.SortedArrayListPriorityQueue<T>`: keep internal ArrayList sorted by priority; insert at correct location
- `chapter9.SortedDLinkedListPriorityQueue<T>`: keep DLinkedList sorted; insert by finding correct node and linking in
- `chapter9.BinaryHeapPriorityQueue<T>` provided: binary heap reference implementation

Benchmark Requirements

You will write a benchmark driver that:

- Uses `System.nanoTime()`
- Has a warmup phase (at least 2 warmup runs per workload)
- Runs multiple trials (recommended 5–9) and reports the median time
- Uses a fixed random seed for reproducibility
- Maintains a **checksum** (e.g., sum of popped/dequeued values) and prints it

Workloads to Measure

Workload	Description	Applies to
W1: Bulk fill then drain	Push/enqueue N items, then pop/dequeue N items.	Stack, Queue, PriorityQueue
W2: Mixed steady-state	Prefill ~10,000 items, then do many random operations: ~60% insert, ~35% remove, ~5% peek.	Stack, Queue, PriorityQueue
W3: Skewed priorities	PriorityQueue only: 90% of priorities in $[0..10]$, 10% in $[11..100000]$.	PriorityQueue

Writeup (1–2 pages)

- Report your timing results (include your printed benchmark output).
- Explain why the winner won for all three interfaces (Stack, Queue, PriorityQueue). In other words, why is the ArrayList impl faster if it was ... or why the DLinkedList impl was faster for each of the operations.
- PriorityQueue: explain why the heap typically dominates for large N and what patterns can make sorted structures look better or worse.

Submission Checklist

- All required classes compile and pass sanity checks
- Your benchmark prints median ns/op and checksum for each workload
- Writeup PDF (or doc) included

Grading

- Correctness: 40%
- Implementation quality: 20%
- Benchmark quality: 25%
- Analysis writeup: 15%